Finding employees with 3rd & 4th highest salary-

Actual ordered table-

```
select * from employees
order by salary desc;
```

Result Grid | Filter Rows: | Edit:

emp_id	name	dept_id	salary	experience
5	Eve	103	90000	8
2	Bob	101	70000	5
7	Grace	101	70000	6
3	Charlie	102	60000	4
4	David	102	60000	3
1	Alice	101	50000	2
6	Frank	103	40000	1
8	Helen	104	NULL	2

Using LIMIT and OFFSET (always write offset after limit otherwise syntax error)-

```
select * from employees
order by salary desc
limit 2
offset 2;
```

Result Grid | Filter Rows: | Edit:

emp_id	name	dept_id	salary	experience
7	Grace	101	70000	6
3	Charlie	102	60000	4
NULL	NULL	NULL	NULL	NULL

With DISTINCT in subquery-

```sql
SELECT *
FROM employees e1
WHERE (
    SELECT COUNT(DISTINCT e2.salary)
    FROM employees e2
    WHERE e2.salary > e1.salary
) IN (2, 3);
```

Result Grid | Filter Rows: | Edit:

emp_id	name	dept_id	salary	experience
1	Alice	101	50000	2
3	Charlie	102	60000	4
4	David	102	60000	3
NULL	NULL	NULL	NULL	NULL

Without DISTINCT in subquery-

```sql
SELECT *
FROM employees e1
WHERE (
    SELECT COUNT(e2.salary)
    FROM employees e2
    WHERE e2.salary > e1.salary
) IN (2, 3);
```

Result Grid | Filter Rows: | Edit:

emp_id	name	dept_id	salary	experience
3	Charlie	102	60000	4
4	David	102	60000	3
NULL	NULL	NULL	NULL	NULL

As you can see, it's very confusing without window functions.

🧠 What does "duplicate salaries" mean?

👉 A salary appears **more than once**

So internally you're thinking:

> "group + count > 1"

✅ Approach 1: Pure Aggregates (simplest & best)

```
SELECT salary, COUNT(*) AS cnt
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1;
```

🧠 Why this works

- `GROUP BY salary` → groups same salaries
- `COUNT(*)` → counts occurrences
- `HAVING COUNT(*) > 1` → keeps duplicates

🔥 If you want full employee rows (not just salary)

Now this is where people get confused.

✅ Approach 2: Join + Aggregate

```
SELECT e.*
FROM employees e
JOIN (
   SELECT salary
   FROM employees
   GROUP BY salary
   HAVING COUNT(*) > 1
) dup
ON e.salary = dup.salary;
```

🧠 Thinking:

1. Inner query:
   → find duplicate salaries
2. Outer query:
   → fetch all employees having those salaries

🟣 Approach 3: Self Join (NO GROUP BY thinking style)

```
SELECT DISTINCT e1.*
FROM employees e1
JOIN employees e2
ON e1.salary = e2.salary
AND e1.id <> e2.id;
```

🧠 Why this works

- Match rows with same salary
- Ensure different rows (`id <> id`)
- Means → duplicate exists

⚠️ Why `DISTINCT`?

Because self-join creates duplicates like:

e1	e2
A	B
B	A

🔥 Pattern Recognition (VERY IMPORTANT)

Whenever you see:

❓ “duplicates”

👉 Your brain should instantly go:

Option A (cleanest)

```
GROUP BY column
HAVING COUNT(*) > 1
```

Option B (if full rows needed)

JOIN with grouped subquery

Option C (if GROUP BY not allowed)

SELF JOIN + id mismatch

🧠 Quick Decision Guide

Requirement	Approach
Just duplicate values	GROUP BY + HAVING
Full rows	JOIN
No aggregates allowed	SELF JOIN

💥 Interview Twist You Might Get

❓ “Find duplicate salaries but show count also”

```
SELECT salary, COUNT(*) AS cnt
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1;
```

🚀 Final Intuition

👉 Duplicate = **“more than one occurrence”**
👉 SQL translation = `COUNT(*) > 1`

INTERVIEW PRACTICE

Tables-

```
CREATE TABLE customers (
   customer_id INT PRIMARY KEY,
   name VARCHAR(50),
   city VARCHAR(50)
);

CREATE TABLE orders (
   order_id INT PRIMARY KEY,
   customer_id INT,
   order_date DATE,
   amount DECIMAL(10,2),
   FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE products (
   product_id INT PRIMARY KEY,
   product_name VARCHAR(50),
   category VARCHAR(50),
   price DECIMAL(10,2)
);

CREATE TABLE order_items (
   order_item_id INT PRIMARY KEY,
   order_id INT,
   product_id INT,
   quantity INT,
   FOREIGN KEY (order_id) REFERENCES orders(order_id),
   FOREIGN KEY (product_id) REFERENCES products(product_id)
);


INSERT INTO customers VALUES
(1, 'Alice', 'Delhi'),
```

(2, 'Bob', 'Mumbai'),
(3, 'Charlie', 'Delhi'),
(4, 'David', 'Bangalore'),
(5, 'Eva', 'Mumbai');

INSERT INTO orders VALUES
(101, 1, '2024-01-10', 500),
(102, 1, '2024-02-15', 300),
(103, 2, '2024-01-12', 700),
(104, 3, '2024-03-05', 200),
(105, 4, '2024-02-20', 1000),
(106, 5, '2024-02-25', 400);

INSERT INTO products VALUES
(1, 'Laptop', 'Electronics', 800),
(2, 'Phone', 'Electronics', 500),
(3, 'Shoes', 'Fashion', 100),
(4, 'Watch', 'Fashion', 200),
(5, 'Tablet', 'Electronics', 600);

INSERT INTO order_items VALUES
(1, 101, 1, 1),
(2, 101, 3, 2),
(3, 102, 2, 1),
(4, 103, 1, 1),
(5, 103, 4, 1),
(6, 104, 3, 1),
(7, 105, 5, 1),
(8, 106, 2, 2);

```
1 •  select * from customers;
```

Result Grid | Filter Rows: | Edit

customer_id	name	city
1	Alice	Delhi
2	Bob	Mumbai
3	Charlie	Delhi
4	David	Bangalore
5	Eva	Mumbai
NULL	NULL	NULL

```
1 •  select * from products;
```

Result Grid | Filter Rows: | Edit:

product_id	product_name	category	price
1	Laptop	Electronics	800.00
2	Phone	Electronics	500.00
3	Shoes	Fashion	100.00
4	Watch	Fashion	200.00
5	Tablet	Electronics	600.00
NULL	NULL	NULL	NULL

```
select * from orders;
```

Result Grid | Filter Rows: | Edit:

order_id	customer_id	order_date	amount
101	1	2024-01-10	500.00
102	1	2024-02-15	300.00
103	2	2024-01-12	700.00
104	3	2024-03-05	200.00
105	4	2024-02-20	1000.00
106	5	2024-02-25	400.00
NULL	NULL	NULL	NULL

```
select * from order_items;
```

Result Grid | Filter Rows: | Edit:

order_item_id	order_id	product_id	quantity
1	101	1	1
2	101	3	2
3	102	2	1
4	103	1	1
5	103	4	1
6	104	3	1
7	105	5	1
8	106	2	2
NULL	NULL	NULL	NULL

🟡 Q1: Total spending per customer

👉 Return:

- customer name
- total amount spent

👉 Concepts:

- JOIN
- GROUP BY

```
select c.customer_id, c.name, sum(o.amount) as total_spending

from customers c

left join orders o

on c.customer_id = o.customer_id

group by c.customer_id, c.name;
```

🟡 Q2: Customers who placed more than 1 order

👉 Return:

- customer_id
- number of orders

👉 Concepts:

- GROUP BY
- HAVING

```
select c.customer_id, count(*) as order_count

from customers c

join orders o

on c.customer_id = o.customer_id

group by c.customer_id

having count(*) > 1;
```

<u>If you want all details-</u>

```
select * from customers c

where c.customer_id in

(select c.customer_id

from customers c

join orders o

on c.customer_id = o.customer_id

group by c.customer_id

having count(*) > 1);
```

```sql
select * from customers c
where c.customer_id in
(select c.customer_id
from customers c
join orders o
on c.customer_id = o.customer_id
group by c.customer_id
having count(*) > 1);
```

Result Grid | Filter Rows: | Edit:

customer_id	name	city
1	Alice	Delhi
NULL	NULL	NULL

🟡 Q3: Top 2 customers by total spending

👉 Concepts:

- ORDER BY
- LIMIT
- Aggregation

select c.customer_id, c.name, sum(o.amount) as total_spending

from customers c

join orders o

on c.customer_id = o.customer_id

group by c.customer_id, c.name

order by total_spending desc

limit 2;

🟡 Q4: Total revenue per city

👉 Concepts:

- JOIN multiple tables
- GROUP BY city

select c.city, sum(o.amount) as total_revenue

from customers c

join orders o

on c.customer_id = o.customer_id

group by c.city;

💡 Minor improvement:

```sql
select c.city, coalesce(sum(o.amount), 0) as total_revenue
from customers c
left join orders o
on c.customer_id = o.customer_id
group by c.city;
```

👉 This includes cities where customers exist but no orders yet (edge case thinking = 🔥)

🟠 Q5: Most popular product (by quantity sold)

👉 Return:

- product_name
- total quantity sold

👉 Concepts:

- JOIN (3 tables)
- GROUP BY
- ORDER BY DESC

```
SELECT p.product_id, p.product_name, SUM(oi.quantity) AS total_quantity

FROM products p

JOIN order_items oi

ON p.product_id = oi.product_id

GROUP BY p.product_id, p.product_name

ORDER BY total_quantity DESC

LIMIT 1;
```

👍 What's correct:

- ✅ Correct aggregation (`SUM(quantity)`)
- ✅ Correct JOIN
- ✅ Correct sorting

⚠️ What's missing (important!):

The question hint said:

> JOIN (3 tables)

You only used **2 tables**.

👉 Why this matters:

- Right now you're counting **all order_items**, but not verifying if they belong to valid orders (in real datasets, this matters)
- Interviewers often expect **complete relational chain**

🔧 Better (interview-safe version):

```
SELECT p.product_name, SUM(oi.quantity) AS total_quantity

FROM products p

JOIN order_items oi ON p.product_id = oi.product_id

JOIN orders o ON oi.order_id = o.order_id

GROUP BY p.product_id, p.product_name

ORDER BY total_quantity DESC;
```

🟠 Q6: Customers who never placed any orders

👉 Concepts:

- LEFT JOIN
- NULL filtering

```
SELECT * FROM customers c

LEFT JOIN orders o

ON c.customer_id = o.customer_id

WHERE o.customer_id IS NULL;
```

🟠 Q7: Average order amount per customer

👉 Concepts:

- GROUP BY
- AVG()

```
SELECT c.customer_id, AVG(amount)

FROM customers c

LEFT JOIN orders o

ON c.customer_id = o.customer_id

GROUP BY c.customer_id;
```

🟠 Q8: Find customers whose total spending is greater than average spending of all customers

👉 ⚠️ Classic interview trap

👉 Concepts:

- Subquery
- GROUP BY
- HAVING

Your query:

SELECT c.customer_id, SUM(o.amount) AS total_spending

FROM customers c

LEFT JOIN orders o

ON c.customer_id = o.customer_id

GROUP BY c.customer_id

HAVING total_spending >

(SELECT AVG(amount) FROM orders);

❌ Problem (important conceptual mistake):

You compared:

👉 `customer total spending`
VS
👉 `average single order amount`

🚫 These are NOT comparable.

🧠 What the question REALLY wants:

Compare each customer's **total spending**
with
average total spending per customer

✅ Correct solution:

```
SELECT c.customer_id, SUM(o.amount) AS total_spending

FROM customers c

JOIN orders o ON c.customer_id = o.customer_id

GROUP BY c.customer_id

HAVING total_spending > (

  SELECT AVG(customer_total)

  FROM (

    SELECT customer_id, SUM(amount) AS customer_total

    FROM orders

    GROUP BY customer_id

  ) t

);
```

🧠 Why this works:

Inner query:

SELECT customer_id, SUM(amount)

👉 gives total per customer

Outer:

AVG(customer_total)

👉 average of those totals

(takes average of total amount spent by each customer instead of taking average of entire summed amount in orders table)

🔥 THIS IS A CORE INTERVIEW PATTERN

Whenever you see:

"greater than average of X"

👉 ALWAYS ask:

- average of rows?
- OR average of grouped values?

🔴 Q9: For each category, find total revenue

👉 Concepts:

- Multi-table JOIN
- GROUP BY category

```
SELECT p.category, SUM(p.price * oi.quantity) AS total_revenue

FROM products p

JOIN order_items oi ON p.product_id = oi.product_id

JOIN orders o ON oi.order_id = o.order_id

GROUP BY p.category;
```

If you want all categories even without any sales, use LEFT JOIN everywhere. If you use LEFT JOIN then JOIN, it eventually becomes both inner join i.e. cancels out previous LEFT JOIN effect.

🔴 Q10: Find the customer who bought the highest number of items (not amount)

👉 ⚠️ Trick:

- SUM(quantity), not amount

```
SELECT c.customer_id, SUM(oi.quantity) AS total_quantity

FROM customers c
```

```
JOIN orders o

ON c.customer_id = o.customer_id

JOIN order_items oi

ON o.order_id = oi.order_id

GROUP BY c.customer_id

ORDER BY total_quantity DESC

LIMIT 1;
```

🔴 Q11: Find repeat customers (ordered in different months)

👉 Return:

- customer_id

👉 Concepts:

- DATE functions
- COUNT DISTINCT

❌ Your first attempt:

```
SELECT c.customer_id, MONTH(o.order_date)

FROM customers c

JOIN orders o

ON c.customer_id = o.customer_id

GROUP BY c.customer_id, MONTH(o.order_date)

HAVING COUNT(*) > 1;
```

👉 This finds:

customers with multiple orders **in the same month**

🚫 Not what we want.

❌ Second attempt:

HAVING COUNT(MONTH(o.order_date)) > 1;

👉 Problem:

- `COUNT()` counts rows, not **distinct months**
- So this is still counting orders, not months

✅ Correct solution:

SELECT c.customer_id

FROM customers c

JOIN orders o

ON c.customer_id = o.customer_id

GROUP BY c.customer_id

HAVING COUNT(DISTINCT MONTH(o.order_date)) > 1;

🧠 KEY PATTERN (VERY IMPORTANT)

Whenever question says:

> "different", "unique", "distinct"

👉 ALWAYS think:

COUNT(DISTINCT column)

🔴 Q12: Find the second highest spending customer

👉 Concepts:

- ORDER BY
- LIMIT with OFFSET

```
SELECT c.customer_id, SUM(o.amount) AS total_spending
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id
ORDER BY total_spending DESC
LIMIT 1 OFFSET 1;
```